PARCOURS PHYSIQUE–CHIMIE

LICENCE 1

SEMESTRE MOUSSON

COURS D'INITIATION A L'INFORMATIQUE

(INF 100)

CHARGE DE COURS

M. WOAMEY

PARTIE 1

NOTIONS SUR L'INFORMATIQUE ET L'ORDINATEUR

Chapitre 1 :

NOTIONS SUR L'INFORMATIQUE

1. ORIGINE DU MOT

Le mot fait sa première apparition en 1957 dans le titre de l'ouvrage de l'ingénieur allemand Karl Steinbuch « *Informatik : Automatische Informationsverarbeitung* ». Mais, le terme français INFORMATIQUE a été employé en 1962 par Philippe Dreyfus. C'est un néologisme français créé à partir des mots « information » et « automatique ». L'Académie Française l'a adopté en 1967 pour nommer la « science du traitement de l'information »

2. DEFINITIONS

a. Information

C'est un élément qui permet de compléter notre connaissance sur une personne, sur un objet ou un environnement. C'est un réel perçu possédant une signification.

b. Informatique

L'informatique est définie simplement comme la science de traitement automatique et rationnel de l'information par une machine automatisée.
Elle peut être aussi perçue comme l'ensemble des activités liées à la conception et à l'emploi des ordinateurs.

3. SYSTEME INFORMATIQUE

C'est l'ensemble des éléments matériels et immatériels utilisés pour le traitement automatique de l'information.

a. Partie matérielle ou hardware

C'est l'ensemble des éléments physiques constituant une machine informatique et les différents supports réseaux ou interconnexion.

b. Partie logicielle ou software

C'est l'ensemble des programmes ou applications utilisés pour le traitement automatique de l'information.

4. QUELQUES DISCIPLINES DE L'INFORMATIQUE

- Informatique théorique (algorithmique, calculabilité, complexité, graphes, langages formels ...)
- Programmation, génie logiciel
- Réseaux, logiciels de télécommunications, sécurité des échanges d'information
- Logiciels de base (systèmes d'exploitation, bases de données, compilateurs ...)
- Systèmes embarqués, robotique
- Images, son, multimédia, interfaces homme/machine, infographie ...
- Systèmes d'information, ingénierie des connaissances ...
- Calcul scientifique, optimisation, intelligence artificielle, bio-informatique, traitement des langues, etc.

5. PETITE LEXIQUE PRATIQUE

- adresse IP : adresse composée de 4 chiffres attribué à chaque ordinateur connecté à internet
- algorithme : ensemble des règles d'opérations relatives à un calcul
- antivirus : logiciel de protection contre les programmes malveillants
- application : outil spécialisé pour une action donnée (exemple : dessin, navigation)
- avatar : image choisie ou créée pour se représenter an niveau d'une communauté
- blog : espace individuel pour s'exprimer publiquement sur internet
- bug : défaut de réalisation qui peut ralentir ou stopper l'activité de l'application
- cheval de Troie : logiciel sous lequel est dissimulé un programme nuisible à votre ordinateur
- clé USB : périphérique pour transporter des données
- disque dur : composant vital où sont stockées les données
- interface : intermédiaire de communication entre l'utilisateur et le programme
- internaute : utilisateur d'internet.
- langage de programmation : jargon conventionnel établi pour créer des programmes informatiques appliquant des algorithmes préformulés
- logiciel : programme informatique facilitant la réalisation d'une tâche
- microprocesseur : pièce considérée comme le cœur de l'ordinateur
- pare-feu : outil de protection contre les tentatives d'intrusions virtuelles (VPN)
- périphérique : équipement extérieur branché à l'ordinateur
- plate-forme : terme désignant une machine virtuelle, un système d'exploitation, un site web, une application ou un logiciel
- programme interactif : programme sollicitant la participation de l'utilisateur

- SPAM : messages électroniques non désirés qui peuvent être filtrés.

Chapitre 2 :

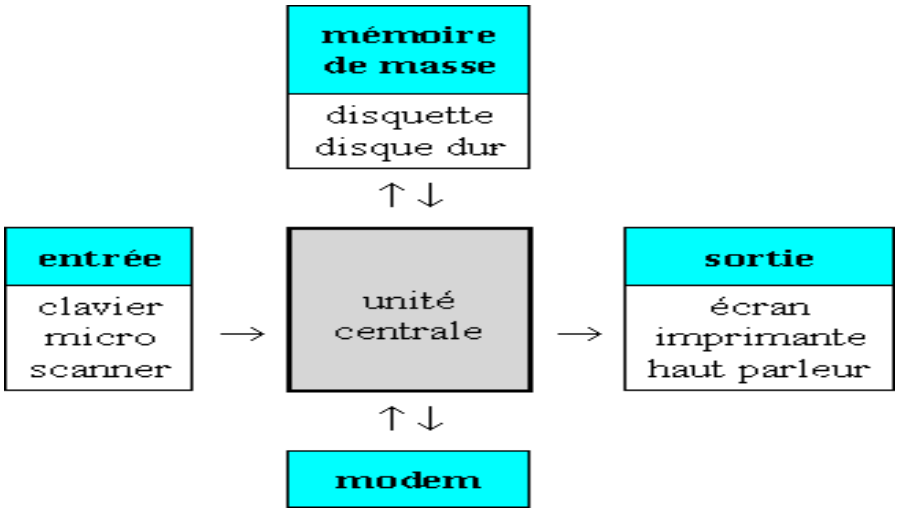
NOTIONS SUR L'ORDINATEUR

1. DEFINITIONS

Un ordinateur est un équipement informatique de traitement automatique de l'information. C'est une machine dotée d'une unité de traitement lui permettant d'exécuter des programmes enregistrés manipulant des données sous forme binaire. L'ordinateur est constitué de deux parties : la partie physique et la partie logicielle. Les éléments physiques permettent à l'utilisateur d'interagir avec les éléments logiciels.

2. DIFFERENTES PARTIES D'UN ORDINATEUR

a. Schéma général des composants



b. Partie matérielle de l'ordinateur

Elle est constituée généralement d'un boitier central (unité centrale) relié à des périphériques d'entrée et de sortie.

- Unité centrale

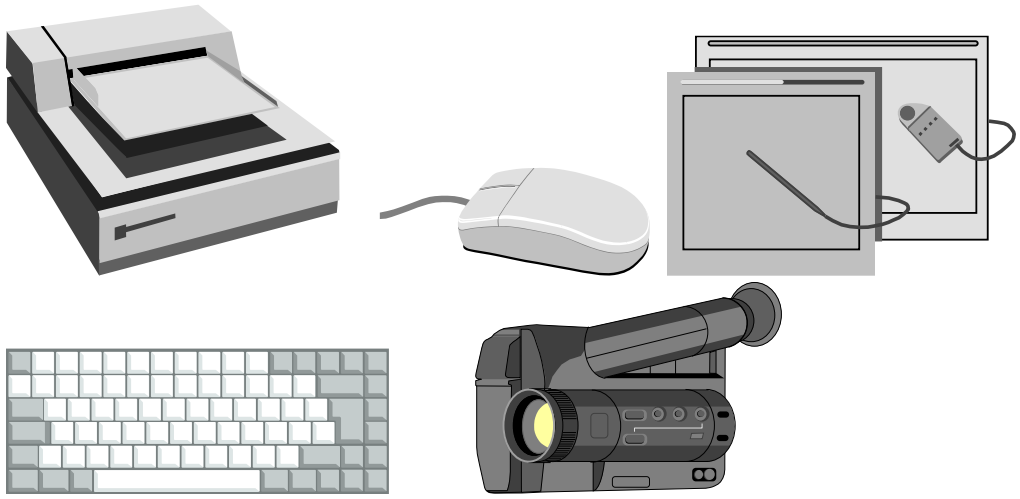
Elle désigne le boitier qui intègre la carte mère sur laquelle est fixé le microprocesseur, la mémoire centrale, le disque dur et d'autres composants. Son élément principal est le microprocesseur. C'est à ce niveau que s'effectuent des traitements automatiques (les opérations sur les bits 1 et 0).

L'unité centrale est constituée de trois parties principales :

- L'unité arithmétique et logique UAL (ou unité de calcul) qui effectue tous les calculs (addition, soustraction, multiplication, etc.).
- L'unité de commande contrôle les opérations sur la mémoire (lecture/écriture) et les opérations à réaliser par l'UAL selon l'instruction en cours d'exécution. Elle est chargée d'interpréter et d'exécuter les opérations en mémoire, et de commander les différents périphériques.
- La mémoire centrale qui permet de stocker momentanément les données à traiter. Dans cette mémoire, nous devons distinguer deux formes de données : la liste des données à traiter et le programme de traitement qui est la liste des instructions que devra exécuter l'ordinateur.

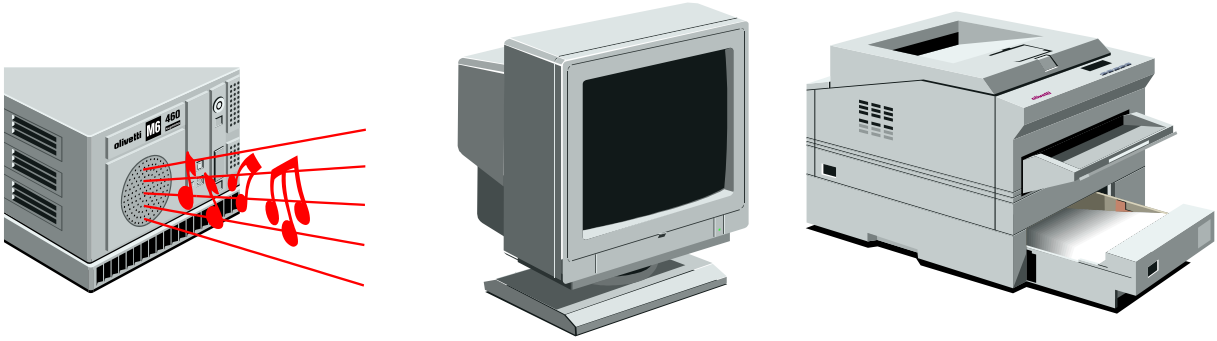
➢ Périphérique d'entrée

Ils servent à commander l'appareil informatique ou à envoyer des informations.
Exemple : le clavier, la souris, le scanner, le micro, la caméra numérique, etc.



➢ Périphérique de sortie

Ils servent à présenter les informations provenant d'un appareil informatique sous une forme reconnaissable à un humain.
Exemple : l'écran ou le moniteur, l'imprimante, le haut-parleur



- Périphérique de masse ou de stockage

Ils servent à conserver les données traitées par un appareil informatique.
Exemple : clé USB, disque CD/DVD, le disque externe, la carte mémoire.



- Périphérique réseau, de connectivité ou d'échange de données entre ordinateurs

Ils servent à connecter plusieurs appareils informatiques.
Exemple : les câbles réseaux, les modems, les switchs ou commutateurs, les routeurs, etc.



c. Partie logicielle de l'ordinateur

Un ordinateur ne peut fonctionner sans la partie logicielle. Un logiciel est un ensemble d'instruction ou de programmes permettant d'effectuer une tâche donnée ou de commander un appareil informatique. On distingue deux sortes de logicielles : les logiciels applicatifs et les logiciels systèmes.

- Les logiciels systèmes

  Un logiciel système contient des instructions ou informations relatives à des opérations routines susceptibles d'être exécuter par plusieurs logiciels applicatifs.
  Un système d'exploitation est un logiciel système qui commande l'utilisation d'un appareil informatique.

Exemple : MICROSOFT DOS, LINUX, UNIX, MS WINDOWS (98, MILLENIUM, XP, VISTA, 7, 8, 10), SUN (MAC OS), ANDROÏD

- Les logiciels applicatifs ou applications informatiques

Un logiciel applicatif contient des instructions ou information relatives à une activité automatisée. En d'autres termes une application est destinée à l'exécution d'une tache donnée.

Les applications peuvent être regroupées en plusieurs types :

- Logiciel de traitement de texte
  Exemple: WORD, EXCEL, POWER POINT, PUBLISHER.

- Les tableurs et les grapheurs
  Exemple : EXCEL, VISIO

- Les logiciels de gestion de base de données
  Exemple : EXCEL, ACCESS, ORACLE, POSTGRE SQL

- Les logiciels de traitement d'images
  Exemple : PHOTOSHOP, ILLUSTRATOR

- Les logiciels ou outils de développement
  Exemple : JAVASCRIPT, PYTHON, PERL, NETBEANS.

3. LES DIFFERENTES FAMILLES D'ORDINATEURS

De nos jours, on a plusieurs différentes familles d'ordinateur.

a. Gros systèmes ou Mainframes

Ce sont des ordinateurs en forme d'armoire ou de bâtiment permettant de faire des calculs sophistiqués complexes et de stocker de grand volume de données.



b. Serveurs

Ils sont les mini-ordinateurs qui permettent de stocker les données ou applications accessibles à plusieurs ordinateurs à travers un réseau informatique. Ils deux formes de serveurs : les serveurs à TOUR et les serveurs en RACK





c. Micro-ordinateurs

Ils se présentent sous deux formes : les DESKTOPS (ordinateurs de bureau) et les LAPTOPS (ordinateurs portables).





d. Autres formes de Laptops : Mini-Laptops, PDA, Smartphones, Tablettes, etc.









4. RESEAU INFORMATIQUE

a. Définition

Un réseau informatique est un ensemble d'équipements reliés entre eux pour échanger des informations.
Les protocoles de communication permettent de définir de façon standardisée la manière dont les informations sont échangées entre les équipements du réseau.
Réseau = équipement + protocoles

b. Différents types de réseaux informatiques.

- Personal Area Network (PAN) ou réseau personnel.
- Local Area Network (LAN) ou réseau local.
- Metropolitan Area Network (MAN) ou réseau métropolitain.
- Wide Area Network (WAN) ou réseau étendu.
- Global Area Network (GAN) ou réseau global.

PARTIE 2

INTRODUCTION A L'ALGORITHMIQUE

Chapitre 1 :

NOTIONS SUR L'ALGORITHMIQUE

1. ALGORITHMIQUE

a. Définition

L'algorithmique est l'ensemble des règles et techniques qui sont appliquées dans la définition et la conception de l'algorithme.

b. Intérêt

Pour faire exécuter une tâche par ordinateur, il faut tout d'abord, détailler suffisamment les étapes de résolution du problème, pour qu'elle soit exécutable par l'homme. Ensuite, transférer la résolution en une suite d'étapes si élémentaire et simple à exécuter, pouvant être codée en un programme dans un langage compréhensible par ordinateur.

Toute suite d'étapes si élémentaire et simple à exécuter s'appelle un algorithme. Un algorithme est une étape préalable à l'écriture d'un programme informatique. Il décrit le résultat de l'analyse d'un problème dans un langage formel. Il se présente sous la forme d'une liste d'opérations permettant de réaliser un travail. L'ordre de ces opérations et leur enchaînement est très important.

2. ALGORITHME ET PROGRAMME

a. Algorithme

Algorithme est une simple déformation latine de *Al Khwarizmi*, le nom du promoteur du concept.

Un algorithme est un énoncé, dans un pseudo-langage défini, d'une suite d'opérations permettant de résoudre par calcul un problème. Il est une suite finie et non ambigüe d'opérations ou d'instructions permettant de donner une réponse à un problème.

La validité d'un algorithme est son aptitude à réaliser exactement la tâche pour laquelle il a été conçu.

La robustesse d'un algorithme est son aptitude à se protéger de conditions anormales d'utilisation.

La réutilisabilité d'un algorithme est son aptitude à être réutilise pour résoudre des tâches équivalentes `a celle pour laquelle il a été conçu.

La complexité d'un algorithme est le nombre d'instructions élémentaires à exécuter pour réaliser la tâche pour laquelle il a été conçu.

L'efficacité d'un algorithme est son aptitude à utiliser de manière optimale les ressources du materiel (en temps et en mémoire) qui l'exécute.

b. Programme

Un programme, c'est une traduction d'un algorithme dans un langage compréhensible par ordinateur à l'aide d'un compilateur. C'est une suite d'opérations prédéterminées pouvant être exécutées par une machine.

Un langage de programmation permet à un humain d'écrire un code pouvant être analysé par une machine puis transformé en un programme informatique.

NB : Le langage PASCAL disposant d'une syntaxe la plus claire et la plus proche du langage dit algorithmique sera utilisé par réécrire les algorithmes de ce cours en programme informatique.

3. ETAPES DE RESOLUTION D'UN PROBLEME

1) Comprendre l'énoncé du problème
2) Décomposer le problème en sous-problèmes plus simple à résoudre
3) Associer à chaque sous problème, une spécification :
   - Les données nécessaires
   - Les données résultantes
   - La démarche à suivre pour arriver au résultat en partant d'un ensemble de données
4) Élaboration d'un algorithme
5) Implémentation d'algorithme en programme par un langage de programmation
6) Exécution de programme pour disposer de résultat au problème.

4. CONSTRUCTION D'UN ALGORITHME

L'écriture d'un algorithme comporte essentiellement trois parties :

a. La partie en-tête

Elle commence par le mot ALGO suivi du nom de l'algorithme. En langage Pascal, elle commence par le mot PROGRAM suivi du nom du programme et est terminée par un point-virgule.

b. Partie déclaration ou définition

C'est dans cette partie où les différentes variables qui seront utilisées dans l'algorithme sont déclarées. On peut aussi de définir dans cette partie des constantes et certains types de données qui pourront être manipulés comme variables dans l'algorithme.

c. Partie corps de l'algorithme

C'est dans cette partie où sont écrites les instructions de l'algorithme. Elle commence par le mot clé *DEBUT* et prend fin par le mot clé *FIN* avec un point-virgule.

d. Structure d'un algorithme

ALGO nom_algorithme
CONST {Définition de constantes}
TYPE {Définition de types}
VAR {Déclaration de variables}
DEBUT
    {Les différentes instructions}
FIN ;

Traduction Pascal
PROGRAM nom_algorithme ;
CONST {Définition de constantes}
TYPE {Définition de types}
VAR {Déclaration de variables}
BEGIN
    {Les différentes instructions}
END.

- *Définition de constantes*

Elle commence par le mot clé *CONST* suivi du nom de la constante et suivi de la valeur de la constante.

CONST nom_constante = valeur_constante ;
Exemple : CONST pi=3,14 ;

- *Déclaration de variables*

Elle commence par le mot clé *VAR* suivi du nom de la variable et du type de la variable.

VAR nom_variable : type_variable ;

Exemple : VAR a : entier ; b, c : reel ;

5. NOTIONS DE VARIABLES ET CONSTANTES

a. Variable

Une variable est un espace mémoire nommé de taille fixe et appartenant à un type déterminé. La valeur contenue dans une variable peut être modifiée. Une variable est l'association d'un nom avec un type, permettant de mémoriser une valeur de ce type.

Le nom d'une variable ne doit comporter que des lettres non accentuées, des chiffres (sauf en première position) et des tirets bas. Il ne peut comporter d'espaces, de signes de ponctuation, de signes arithmétiques ou de caractères spéciaux.

b. Constante

Une constante est un espace mémoire nommé de valeur non modifiable. La valeur de la constante permet de connaitre son type. Une constante a une valeur définie au début du programme et qui reste inchangée durant toute son exécution.

6. TYPES DE DONNEES

Un type de données est un ensemble de valeurs qu'une variable peut prendre. On distingue des types élémentaires et des types structurés (tableau, enregistrement, énumération, etc.). Il faut noter qu'à un type donné, correspond un ensemble d'opérations définies pour ce type.

a. Type élémentaire

Le type entier

Le type Entier caractérise les entiers relatifs. Les opérations utilisables sur les entiers sont :

- Les opérateurs arithmétiques classiques : + (addition), - (soustraction), * (produit)
- La division entière, notée DIV, telle que n DIV p donne la partie entière du quotient de la division de n par p
- Le modulo, notée MOD, telle que n MOD p donne le reste de la division entière de n par p
- Les opérateurs de comparaison classiques : $<$, $>$, $=$, $\neq$, $\geq$, $\leq$

Le type réel

Les opérations utilisables sur les réels sont :

- Les opérateurs arithmétiques classiques : + (addition), - (soustraction), * (produit), / (division)
- Les opérateurs de comparaison classiques : $<$, $>$, $=$, $\neq$, $\geq$, $\leq$

Le type booléen

Il s'agit du domaine dont les seules valeurs sont vrai ou faux. Les opérations utilisables sur les booléens sont réalisées à l'aide des connecteurs logiques : ET (pour le ET logique), OU (pour

le OU logique), NON (pour le NON logique). Les opérateurs de comparaison = et ≠ peuvent être également utilisés pour les booléens.

Le type caractère

Il s'agit du domaine constitué des caractères alphabétiques et numériques. Les opérations élémentaires réalisables sont les comparaisons : <, >, =, ≠, ≥, ≤.

Le type chaîne

Une chaîne est une séquence de plusieurs caractères. Les opérations élémentaires réalisables sont les comparaisons : <, >, =, ≠, ≥, ≤ selon l'ordre lexicographique. L'opérateur (+) de concaténation est aussi possible pour relier bout en bout deux chaines.

Le tableau suivant récapitule ces types élémentaires de données avec leurs opérations.

Type	**Domaines de valeurs**	**Opérateurs applicables**
Entier / Réel	**Z** / $\mathbb{R}$	$+, -, \div, *, \leq, \geq, <, >, \neq, =$
Caractère	'a', 'b', 'A', 'B'	$\leq, \geq, <, >, \neq, =$
Booléen	{Vrai, Faux}	Et, ou, non, $\neq$, $=$
Chaine de caractère	'bonjour', 'BONJOUR'	$<$, $>$, $\leq$, $\geq$, $\neq$, $=$, $+$ **(***Concaténation***)**

b. Type Intervalle

Les types intervalles sont très utilisés. Ils peuvent être de n'importe quel type scalaire. Un intervalle est forcément ordonné et continu.

Syntaxe

TYPE mon_type = borne_inf..borne_sup ;

Exemple

TYPE Binaire = 0..1 ;
TYPE Alpha = 'A'..'Z' ;
TYPE Cent = 1..100 ;

c. Type Enuméré

Un type énuméré est un type dont les variables associées n'auront qu'un nombre très limité de valeurs (au maximum 256 différentes possibles). La définition d'un type énuméré consiste à déclarer une liste de valeurs possibles associées à un type.

Exemple

TYPE Jours = (dim, lun, mar, mer, jeu, ven, sam) ;

d. Type Enregistrement

Un type enregistrement est un type structuré ou composite comportant des éléments de différents sous-types élémentaires ou non. Pour déclarer une variable de type enregistrement, il faut d'abord définir le type enregistrement.

Syntaxe

```
TYPE  nom_type_structure = STRUCT {
                                    champ1 : type1;
                                    champ2 : type2;
                                    champ3 : type3;
                                    . . . . . .
                                    champN : typeN;
                                    } ;
```

Traduction en Pascal

```
TYPE nom_du_type = RECORD
                        champ1 : type1;
                        champ2 : type2;
                        champ3 : type3;
                        . . .
                        champN : typeN ;
                    END ;
```

Exemple

```
TYPE Identite = RECORD
                    Nom : String[20];
                    Age : Byte;
                    Sexe : Char;
                END ;
```

e. Type Tableau

Un type tableau est un type structuré permettant de définir une collection finie et ordonnée de valeurs toutes de même type. La collection est ordonnée et chaque valeur est repérable par sa position au sein de la collection : indice.

Ce type de données sera étudié en détail ultérieurement.

Exercices : QCM (un seul item correct par question)

1. L'informatique est la science
(a) des dispositifs dont le fonctionnement dépend de la circulation d'électrons
(b) des signaux électriques porteurs d'information ou d'énergie
(c) du traitement automatique de l'information
(d) de la commande des appareils fonctionnant sans intervention humaine

2. Le logiciel est
(a) la mémoire de l'ordinateur
(b) le traitement automatique de l'information
(c) l'ensemble des données manipulées par les instructions
(d) un ensemble structuré d'instructions décrivant un traitement d'informations à faire réaliser par un matériel informatique

3. L'algorithmique est la science
(a) du traitement automatique de l'information
(b) des algorithmes
(c) des langages de programmation
(d) des instructions

4. Un algorithme est
(a) un ensemble de programmes remplissant une fonction déterminée, permettant l'accomplissement d'une tâche donnée
(b) une suite ordonnée d'instructions qui indique la démarche à suivre pour résoudre une série de problèmes équivalents
(c) le nombre d'instructions élémentaires à exécuter pour réaliser une tâche donnée
(d) un ensemble de dispositifs physiques utilisés pour traiter automatiquement des informations

5. La validité d'un algorithme est son aptitude
(a) à utiliser de manière optimale les ressources du matériel qui l'exécute
(b) à se protéger de conditions anormales d'utilisation
(c) à calculer le nombre d'instructions élémentaires nécessaires pour réaliser la tâche pour laquelle il a été conçu
(d) à réaliser exactement la tâche pour laquelle il a été conçu

Chapitre. 2 :

LES INSTRUCTIONS

1. DEFINITION

Une instruction est une action élémentaire commandant à la machine un calcul, ou une communication avec l'un de ses périphériques d'entrées ou de sorties. Elle traduit une ou plusieurs actions portant sur une ou plusieurs variables. Ces actions sont relatives aux opérations admissibles sur les valeurs des variables.

2. INSTRUCTIONS DE BASE

a. Instruction d'entrée ou de saisie

Une instruction d'entrée autorise ou exécute la saisie d'information à partir d'un périphérique d'entrée.

LIRE (nom_variable) ;

LIRE (a) ; signifie qu'on affecte la valeur ou l'information saisie au clavier par l'utilisateur à la variable nommée a.

En Pascal, on utilise le mot-clé *READ* ou *READLN* suivi de la ou les variables entre parenthèses.

b. Instruction de sortie ou d'affichage

Une instruction de sortie affiche le contenu d'une variable, d'une constante ou une information sur périphérique (*écran par exemple*).

ECRIRE (' une information ') ;
ECRIRE (nom_constante) ;
ECRIRE (nom_variable) ;
ECRIRE (' information ', nom_variable) ;
ECRIRE (un_résultat) ;

Exemple

ECRIRE ('Bonjour') ; signifie qu'on va afficher à l'écran la chaine de caractères **Bonjour.**

ECRIRE (a) ; signifie qu'on va afficher à l'écran le contenu de la variable ou de la constante **a**.

*ECRIRE ('La valeur de 3*5 est ', 3*5)* ; permet d'afficher sur l'écran la phrase suivante : La valeur de 3*5 est 15. L'expression entre apostrophes s'affiche comme telle et l'expression 3*5 après la virgule est évaluée et son résultat affiché.

***NB** : Théoriquement, toute instruction de lecture doit être précédée d'une instruction d'écriture informative.*

En Pascal, on utilise le mot-clé *WRITE* suivi de l'expression entre parenthèses, ou des expressions séparées par des virgules. Une chaine de caractère doit être délimitée par des apostrophes. Pour passer à la ligne après l'affichage, il faut remplacer *WRITE* par *WRITELN*.

Exercice d'application

Ecrire un algorithme qui demande à l'utilisateur son nom, son prénom et son âge et qui lui affiche après ces informations sur son identité.

Solution

```
ALGO Identite
VAR    N : chaine ;
       P : chaine ;
       A : entier ;
DEBUT
       ECRIRE ('Votre Nom') ;
       LIRE (N) ;
       ECRIRE ('Votre prénom') ;
       LIRE (P) ;
       ECRIRE ('Votre âge') ;
       LIRE (A) ;
       ECRIRE ('Votre nom est :  ', N) ;
       ECRIRE ('Votre prénom est :  ', P) ;
       ECRIRE ('Votre âge est :  ', A) ;
FIN ;
```

Traduction Pascal

```
PROGRAM Identite ;
VAR    N : String ;
       P : String ;
       A : Integer ;
BEGIN
       WRITELN ('Votre Nom') ;
       READLN (N) ;
       WRITELN ('Votre prénom') ;
       READLN (P) ;
       WRITELN ('Votre âge') ;
       READLN (A) ;
```

WRITELN ('Votre nom est : ', N) ;
WRITELN ('Votre prénom est : ', P) ;
WRITELN ('Votre âge est : ', A) ;
END.

c. Instruction d'affectation

Elle permet d'affecter à une variable le contenu d'une autre variable, le contenu d'une constante, d'une valeur fixe ou le résultat d'un calcul.

var 1← var 2 ; signifie *la variable var 1 reçoit le contenu de la variable var 2*
var 1← const 1 ;
var 1←valeur 1 ;
var 1←res 1 ;

En Pascal, elle est symbolisée par le signe **:=**

Exercice d'application

Ecrire un algorithme qui demande à l'utilisateur deux réels quelconques non nuls et qui lui affiche leur somme, leur différence, leur produit et leur rapport.

Solution

ALGO Calcul
VAR a, b, S, P, D, R : reel ;
DEBUT
ECRIRE ('saisir deux réels non nuls') ;
LIRE (a, b) ;
S←a+b ;
P←a∗ b;
D← a − b;
R← a/b;
ECRIRE ('leur somme est ', S) ;
ECRIRE ('leur produit est ', P) ;
ECRIRE ('leur différence est ', D) ;
ECRIRE ('leur rapport est ', R) ;
FIN ;

Traduction Pascal
PROGRAM Calcul;
VAR a, b, S, P, D, R : Real ;
BEGIN
WRITELN ('saisir deux réels non nuls') ;
READLN (a, b) ;
S := a+b ;

P := a∗ b;
D := a − b;
R := a/b;
WRITELN ('leur somme est ', S) ;
WRITELN ('leur produit est ', P) ;
WRITELN ('leur différence est ', D) ;
WRITELN ('leur rapport est ', R) ;
END.

d. Les expressions arithmétiques

Une expression est une suite finie bien formée d'opérateurs portant sur des variables ou des valeurs et qui a une valeur. La valeur de l'expression doit être conforme au domaine de la variable affectée. Les expressions arithmétiques utilisent les fonctions et les opérateurs.

➢ Les fonctions

- La fonction ***DIV*** permet de donner le résultat de la division entière d'un nombre par un autre. *7 DIV 2 donne 3.*
- La fonction ***MOD*** (se lit Modulo), permet de donner le reste de la division entière d'un entier par un autre. *7 MOD 2 donne 1.*
- La fonction ^ permet d'élever un nombre à la puissance d'un autre. *2 ^ 3 donne 8.*
  <u>NB</u> : L'algorithme qui sous-tend cette fonction sera écrit par la suite.

➢ Les opérateurs

Les opérateurs arithmétiques sont le +, -, /, *.
Ces opérateurs sont ordonnés du plus prioritaire au moins prioritaire dans l'évaluation d'une expression arithmétique.
1- Les parenthèses
2- Les fonctions
3- Les opérateurs de multiplication " * " et de division " / "
4- Les opérateurs d'addition " + " et de soustraction " - "

<u>Remarque</u>
Si l'ordre entre les opérateurs dans une expression est le même, on évalue l'expression de gauche à droite.

<u>Exemples</u>
3 ^ 2 + 4 = 9 + 4 = 13
3 ^ (2 + 4) =3 ^ 6 car les parenthèses sont plus prioritaires
17 MOD 10 DIV 3 = (17 MOD 10) DIV 3 = 7 DIV 3 = 2

<u>Exercices 1</u> : QCM (un seul item correct par question)

1. Une variable informatique est un objet

(a) équivalent à une variable mathématique
(b) qui associe un nom à une valeur
(c) qui varie nécessairement
(d) qui modifie la mémoire

2. L'affectation consiste à
(a) comparer la valeur d'une variable à une autre valeur
(b) associer une valeur à une variable
(c) incrémenter une variable
(d) déplacer une variable en mémoire

3. Après la séquence

a ← 13 ;
b ← 4 ;
b ← a ;
a ← b ;

les variables a et b sont telles que
(a) a = 13 et b = 13
(b) a = 4 et b = 4
(c) a = 4 et b = 13
(d) a = 13 et b = 4

Exercices 2

1) Ecrire un algorithme qui permute la valeur de deux variables c1 et c2 de type caractère.

2) Ecrire un algorithme qui demande à l'utilisateur de saisir deux réels non nuls et qui effectue la somme du triple du premier nombre avec la moitié du deuxième et affiche le résultat à l'utilisateur.

3) $f(x,y) = \frac{x^2}{2x-y}$ .Ecrire un algorithme qui demande à l'utilisateur de donner la valeur des inconnus et qui lui affiche le résultat.

4). Écrire un algorithme qui permet de déterminer et d'afficher la position (x, y), à l'instant t, d'un mobile en mouvement dans un plan donné par les équations paramétriques suivantes :

$$\begin{cases} x = 2t \\ y = \frac{3t^2}{2} - 4t \end{cases}$$

3. INSTRUCTIONS CONDITIONNELLES

Une instruction conditionnelle permet d'exécuter ou non un bloc d'instructions selon le résultat d'un test (valeur booléenne).

a. Structure SI…ALORS ou SI INCOMPLET

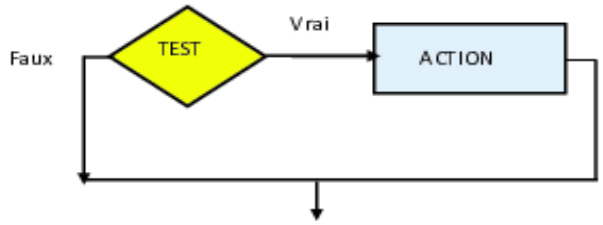
Ce type de structure présente une seule possibilité ; elle ne s'exécute que dans le cas où la condition est vérifiée. Le cas contraire est ignoré.

Syntaxe

SI (condition vérifiée) ALORS
    Instruction ;
FIN SI ;

Traduction Pascal
IF (condition vérifiée) THEN Instruction ;
NB : Si Instruction est une suite d'instructions, on l'encadre par BEGIN et END.

Organigramme



Exemple :

Ecrire un algorithme qui demande à l'utilisateur de saisir un réel et qui lui affiche un réel positif si le réel saisi est positif.

Solution

```
ALGO TEST_POSITIF
VAR   n : reel ;
DEBUT
        ECRIRE ('Saisir un réel') ;
        LIRE (n) ;
        SI (n> 0) ALORS
                ECRIRE ('Réel positif') ;
        FIN SI ;
FIN ;
```

Traduction Pascal

```
PROGRAM TEST_POSITIF;
VAR    n : Real ;
BEGIN
        WRITELN ('Saisir un réel') ;
        READLN (n) ;
        IF (n> 0) THEN WRITELN ('Réel positif') ;
END.
```

b. Structure SI…ALORS…SINON ou SI COMPLET

Cette instruction offre un choix entre deux possibilités, selon le résultat du test de la condition.

Syntaxe

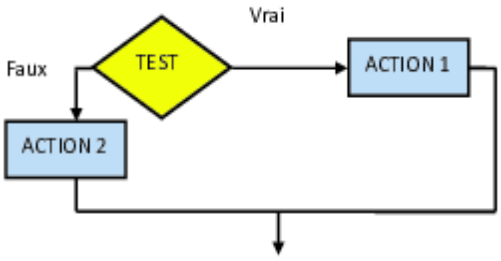
```
SI (condition vérifiée) ALORS
    Instruction 1 ;
SINON
    Instruction 2 ;
FIN SI ;
```

Traduction Pascal

```
IF (condition vérifiée) THEN      Instruction 1
ELSE       Instruction 2 ;
```

NB : Si Instruction1 ou Instruction2 est une suite d'instructions, on l'encadre par BEGIN et END.

Organigramme



Exemple

```
ALGO TEST_POSITIF
VAR    n : reel ;
DEBUT
        ECRIRE ('Saisir un réel') ;
        LIRE (n) ;
```

```
        SI (n> 0) ALORS
                ECRIRE ('Réel positif') ;
        SINON
                ECRIRE ('Réel non positif') ;
        FIN SI ;
FIN ;
```

Traduction Pascal

```
PROGRAM  TEST_POSITIF ;
VAR    n : Real ;
BEGIN
        WRITELN ('Saisir un réel') ;
        READLN (n) ;
        IF (n> 0) THEN WRITELN ('Réel positif')
        ELSE WRITELN ('Réel non positif') ;
END.
```

c. Structure SI…ALORS…SINON SI ou SI IMBRIQUE

Le traitement de certains problèmes qui présentent plusieurs possibilités implique l'ouverture de plusieurs voies, correspondant à des tests imbriqués les uns dans les autres.

Syntaxe

```
SI (condition 1 vérifiée) ALORS
    Instruction 1 ;
SINON   SI (condition 2 vérifiée) ALORS
                Instruction 2 ;
        SINON       SI (condition 3 vérifiée) ALORS
                            Instruction 3 ;
                    SINON
                            Instruction 4 ;
                    FIN SI
        FIN SI
FIN SI ;
```

Traduction Pascal

```
IF (condition 1 vérifiée) THEN Instruction 1
ELSE IF (condition 2 vérifiée) THEN Instruction 2
ELSE IF (condition 3 vérifiée) THEN Instruction 3
ELSE Instruction 4 ;
```

Exemple :

Ecrire un algorithme qui demande à l'utilisateur un nombre réel donné et qui lui affiche réel positif si le réel est positif, réel négatif si le réel est négatif et réel nul si le réel saisi est nul

Solution

```
ALGO TEST_REEL
VAR   n : reel ;
DEBUT
        ECRIRE ('Saisir un réel') ;
        LIRE (n) ;
        SI (n>0)  ALORS
                ECRIRE ('Réel positif') ;
        SINON  SI (n< 0) ALORS
                        ECRIRE ('Réel négatif ') ;
                SINON
                        ECRIRE ('Réel nul') ;
                FIN  SI
        FIN  SI ;
FIN ;
```

4. INSTRUCTION ALTERNATIVE : SELON

L'instruction SELON permet d'éviter plusieurs imbrications de conditionnelles, et permet de traiter directement des choix ou alternatifs. Elle est réservée à des conditions bien précises (pour les variables de type entier, caractère, chaine, énumération).

Syntaxe

```
SELON variable / expression
        Valeur1 / expression1 : instruction1 ;
        Valeur2 / expression2 : instruction2 ;
        . . .
        Valeur_n / expression_n : instruction_n ;
        SINON instruction par défaut ;
FIN SELON ;
```

Traduction Pascal

```
CASE variable / expression OF
        Valeur1 / expression1 : instruction1 ;
        Valeur2 / expression2 : instruction2 ;
        . . .
        Valeur_n / expression_n : instruction_n
```

ELSE instruction par défaut ;
END ;

Exemple

Ecrire un algorithme qui donne le nombre de jours dans un mois donné.

Solution

```
ALGO NOMBREJOURS_MOIS
VAR   mois : entier ;
DEBUT
        ECRIRE ('Saisir la valeur du mois comme entier') ;
        LIRE (mois) ;
        SELON mois
                1 : ECRIRE ('31 jours pour Janvier') ;
                2 : ECRIRE ('28 ou 29 jours pour Février') ;
                3 : ECRIRE ('31 jours pour Mars') ;
                4 : ECRIRE ('30 jours pour Avril') ;
                5 : ECRIRE ('31 jours pour Mai') ;
                6 : ECRIRE ('30 jours pour Juin') ;
                7 : ECRIRE ('31 jours pour Juillet') ;
                8 : ECRIRE ('31 jours pour Août') ;
                9 : ECRIRE ('30 jours pour Septembre') ;
                10 : ECRIRE ('31 jours pour Octobre') ;
                11 : ECRIRE ('30 jours pour Novembre') ;
                12 : ECRIRE ('31 jours pour Décembre') ;
                SINON ECRIRE ('Ce chiffre ne correspond pas à la valeur d'un mois')
        FIN SELON ;
FIN ;
```

Traduction Pascal

```
PROGRAM NOMBREJOURS_MOIS ;
VAR   mois : Integer ;
BEGIN
        WRITELN ('Saisir la valeur du mois comme entier') ;
        READLN (mois) ;
        CASE mois OF
                1 : WRITELN ('31 jours pour Janvier') ;
                2 : WRITELN ('28 ou 29 jours pour Février') ;
                3 : WRITELN ('31 jours pour Mars') ;
                4 : WRITELN ('30 jours pour Avril') ;
                5 : WRITELN ('31 jours pour Mai') ;
                6 : WRITELN ('30 jours pour Juin') ;
```

```
                7 : WRITELN ('31 jours pour Juillet') ;
                8 : WRITELN ('31 jours pour Août') ;
                9 : WRITELN ('30 jours pour Septembre') ;
                10 : WRITELN ('31 jours pour Octobre') ;
                11 : WRITELN ('30 jours pour Novembre') ;
                12 : WRITELN ('31 jours pour Décembre')
                ELSE WRITELN ('Ce chiffre ne correspond pas à la valeur d'un mois')
        END ;
END.
```

Exercices : QCM (un seul item correct par question)

1. Le résultat d'une comparaison est une valeur
(a) réelle
(b) qui dépend du type des arguments
(c) booléenne
(d) entière

2. Un operateur booléen s'applique à des valeurs
(a) booléennes
(b) entières
(c) réelles
(d) alphanumériques

3. La fonction principale d'une instruction de test est
(a) de passer d'instruction en instruction
(b) de répéter une instruction sous condition
(c) d'exécuter une instruction sous condition
(d) d'interrompre l'exécution d'une instruction

5. LES INSTRUCTIONS REPETITIVES

Une instruction répétitive permet d'effectuer un traitement à plusieurs reprises.

a. Structure POUR … FAIRE

Elle permet d'exécuter une séquence d'instructions un nombre de fois connu à priori. Cette répétition se fait grâce à une variable compteur ou indice qui progresse dans un intervalle, pris sur un ensemble dénombrable et ordonné (entiers, char).

Syntaxe

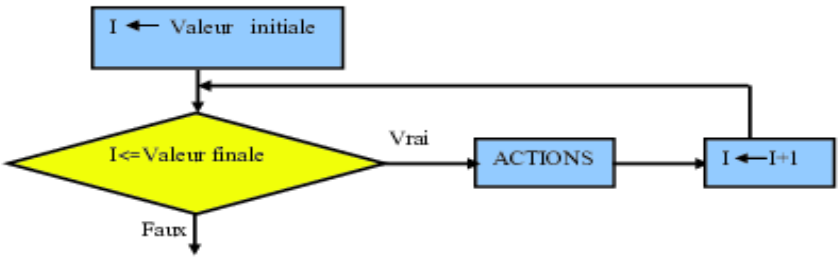
POUR *compteur* DE *valeur1* A *valeur2 [PAS valeur_pas]* FAIRE
    {suite d'instructions} ;
FIN POUR ;

Traduction Pascal
FOR *compteur* := *valeur_inférieure* TO *valeur_supérieure DO*
BEGIN
    {suite d'instructions} ;
END ;

ou

FOR *compteur* := *valeur_supérieure* DOWNTO *valeur_inférieure DO*
BEGIN
    {suite d'instructions} ;
END ;

Organigramme



NB :

Par défaut, la valeur de pas est 1 ; ainsi la partie à crochets est ignorée. Aussi, si valeur1>valeur2, on parlera d'une décrémentation en lieu et place d'incrémentation.

Exercices d'application 1

1) Ecrire un algorithme qui fait la somme des milles premiers entiers naturels non nuls.

2) Ecrire un algorithme qui fait le produit des dix premiers entiers naturels non nuls.

3) Ecrire un algorithme qui fait la somme des carrées des cents premiers entiers naturels.

4) Ecrire un algorithme qui étant donné trois réels saisis par l'utilisateur lui renvoi leur minimum.

5) Ecrire un algorithme qui étant donné quatre réels saisis par l'utilisateur lui renvoi leur maximum.

6) Ecrire un algorithme qui donne la valeur absolue d'un réel saisi par l'utilisateur.

Correction Exo1

```
ALGO SOMME
VAR    n, S : entier ;
DEBUT
        S← 0 ;
        POUR n DE 1 A 1000 FAIRE
                S← S + n ;
        FIN POUR ;
        ECRIRE (' la somme des 1000 entiers naturels non nuls est ', S) ;
FIN ;
```

Traduction Pascal

```
PROGRAM SOMME;
VAR    n, S : Integer ;
BEGIN
        S := 0 ;
        FOR n := 1 TO 1000 DO
        BEGIN
                S := S + n ;
        END ;
        WRITELN (' la somme des 1000 entiers naturels non nuls est ', S) ;
END. ;
```

Correction Exo2

```
ALGO PRODUIT
VAR    i, P : entier ;
DEBUT
        P← 1;
        POUR i DE 1 A 10 FAIRE
                P← P * i ;
        FIN POUR ;
        ECRIRE ('Le produit des 10 premiers entiers naturels non nuls est ', P) ;
```

FIN ;

Correction Exo3

```
ALGO SOMME
VAR   i, S : entier ;
DEBUT
        S← 0 ;
        POUR  i DE 1 A 100 FAIRE
                S← S + (i ∗ i);
        FIN POUR ;
        ECRIRE ('La somme des carrées des 100 premiers entiers naturels est', S ) ;
FIN ;
```

Correction Exo4

```
ALGO MINIMUM
VAR   a, b, c, Min : reel ;
DEBUT
        ECRIRE (' Saisir trois réels') ;
        LIRE (a, b, c) ;
        Min ← a ;
        SI (b<Min) ALORS
                Min ← b ;
        FIN SI ;
        SI (c< Min) ALORS
                Min← c;
        FIN SI ;
        ECRIRE ('Le minimum des trois réels ', a,' ', b, ' et ', c, ' est ', Min) ;
FIN ;
```

Traduction Pascal

```
PROGRAM MINIMUM;
VAR   a, b, c, Min : Reall ;
BEGIN
        WRITELN (' Saisir trois réels') ;
        READLN (a, b, c) ;
        Min := a ;
        IF (b < Min) THEN Min := b ;
        IF (c < Min) THEN Min := c;
        WRITELN ('Le minimum des trois réels ', a,' ', b, ' et ', c, ' est ', Min) ;
END.
```

Correction Exo5

```
ALGO MAXIMUM
VAR    a, b, c, d, Max : reel ;
DEBUT
        ECRIRE ('Saisir 4 réels') ;
        LIRE (a, b, c, d) ;
        Max← d ;
        SI (c> Max) ALORS
                Max← c ;
        FIN SI ;
        SI (b> Max) ALORS
                Max← b ;
        FIN SI ;
        SI (a> Max) ALORS
                Max← a ;
        FIN SI ;
        ECRIRE ('Le maximum des 4 réels est ', Max) ;
FIN ;
```

Correction Exo6

```
ALGO VALEUR_ABSOLUE
VAR   x, v : reel ;
DEBUT
        ECRIRE ('Saisir x ') ;
        LIRE (x) ;
        SI (x≥ 0) ALORS
                v← x ;
        FIN SI ;
        SI (x< 0) ALORS
                v← (−1) ∗ (x) ;
        FIN SI ;
        ECRIRE ('La valeur absolue de ', x, ' est ', v) ;
FIN ;
```

Exercices d'application 2

1) Ecrire un algorithme qui donne la puissance nième d'un nombre réel saisi par l'utilisateur (n ∈ ℕ).

2) Soit un nombre entier naturel n, Ecrire un algorithme qui demande à l'utilisateur de préciser la valeur de n et qui lui affiche le produit et la somme des n premiers entiers.

Correction

```
ALGO PUISSANCE
VAR   P, a : reel ;
      i, n : entier ;
DEBUT
      ECRIRE ('Saisir un nombre réel') ;
      LIRE (a) ;
      ECRIRE ('Saisir l'exposant n) ;
      LIRE (n) ;
      P← 1 ;
      POUR i DE 1 à n FAIRE
            P ← P * a ;
      FIN POUR ;
      ECRIRE ('La puissance de ', a, ' exposant ', n, ' est ', P) ;
FIN ;
```

b. Structure TANT QUE ... FAIRE

Dans cette structure, le bloc d'instructions n'est exécuté qu'après la vérification de la condition.

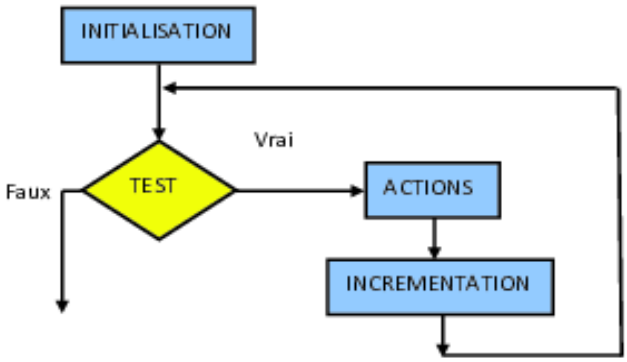
Syntaxe

```
Initialisation_compteur ;
TANT QUE (condition_vérifiée) FAIRE
      Instructions ;
      Compteur_suivant ;
FIN TANT QUE ;
```

Traduction Pascal

```
Initialisation_compteur ;
WHILE (condition_vérifiée) DO
BEGIN
      Instructions ;
      Compteur_suivant ;
END ;
```

Organigramme



Exemple

Ecrire un algorithme qui fait la somme des 5 premiers entiers positifs saisi par l'utilisateur et qui lui affiche le résultat.

Solution

```
ALGO SOMME
VAR  S, n, i : entier ;
DEBUT
        S← 0 ;
        i← 0 ;
        TANT QUE (i < 5) FAIRE
                ECRIRE ('Entrez un entier positif') ;
                LIRE (n) ;
                SI (n > 0) ALORS
                        i← i + 1 ;
                        S← S + n ;
                FIN SI ;
        FIN TANT QUE ;
        ECRIRE ('La somme est ', S) ;
FIN ;
```

Traduction Pascal

```
PROGRAM SOMME;
VAR  S, n, i : Integer ;
BEGIN
        S := 0 ;
        i := 0 ;
        WHILE (i < 5) DO
        BEGIN
                WRITELN ('Entrez un entier positif') ;
                READLN (n) ;
```

```
            IF (n > 0) THEN
            BEGIN
                  i := i + 1 ;
                  S := S + n ;
            END ;
      END ;
      WRITELN ('La somme est ', S) ;
END.
```

Exercice d'application :

Ecrire un algorithme qui effectue le produit des carrées de cinq entiers positifs et non nuls saisis par l'utilisateur et qui affiche les différents entiers et le résultat du produit

c. Structure REPETER … JUSQU'A

Cette boucle permet de répéter les instructions jusqu'à la vérification de la condition.

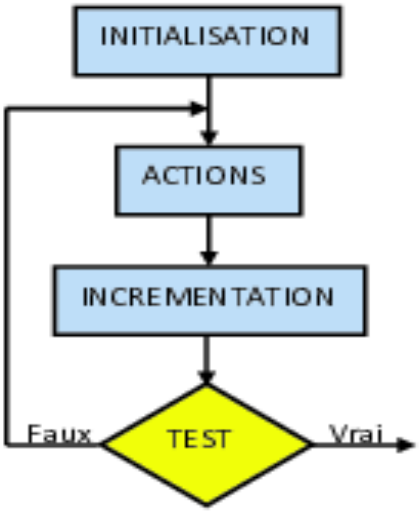
Syntaxe

```
Initialisation_compteur ;
REPETER
      Instructions ;
      Compteur_suivant ;
JUSQU'A (condition_d'arrêt)
FIN REPETER ;
```

Traduction Pascal

```
Initialisation_compteur ;
REPEAT
      Instructions ;
      Compteur_suivant ;
UNTIL (condition_d'arrêt) ;
```

Organigramme



Exemple

ALGO PUISSANCE
VAR m, p : reel ;
i, n : entier ;
DEBUT
ECRIRE ('entrer un nombre réel') ;
LIRE (m) ;
ECRIRE ('entrer un entier comme exposant') ;
LIRE (n) ;
P$\leftarrow 1$;
i$\leftarrow 0$;
REPETER
$p \leftarrow p * m$;
$i \leftarrow i + 1$;
JUSQU'A $(i = n)$
FIN REPETER ;
ECRIRE ('la puissance est ', p) ;
FIN **;**

Traduction Pascal
PROGRAM PUISSANCE ;
VAR m, p : Real ;
i, n : Integer ;
BEGIN
WRITELN ('entrer un nombre réel') ;
READLN (m) ;
WRITELN ('entrer un entier comme exposant') ;

```
        READLN (n) ;
        p := 1 ;
        i := 0 ;
        REPEAT
                p := p * m ;
                i := i + 1 ;
        UNTIL (i = n) ;
        WRITELN ('la puissance est ', p) ;
END.
```

d. La différence entre la boucle Tant que et Répéter

- Les instructions de la boucle Répéter sont exécutées au moins une fois, car l'évaluation de la condition vient après le passage dans la boucle. Cependant, les instructions de la boucle Tant que peuvent ne jamais être exécutées si la condition n'est pas vérifiée, lors de la première évaluation.
- La boucle Répéter s'exécute jusqu'à ce que la condition soit vérifiée. Donc la condition permet la sortie de la boucle Répéter. La boucle Tant que s'exécute tant que la condition est vérifiée. Donc la condition permet le passage dans la boucle Tant que.
- Une séquence d'instructions nécessite une délimitation par Début . . . Fin dans le cas d'une boucle Tant que ; ce qui n'est pas le cas de la boucle Répéter.

e. Comment choisir le type de boucle à utiliser ?

- Si on peut déterminer le nombre d'itérations avant l'exécution de la boucle, il est plus judicieux d'utiliser la boucle Pour.
- S'il n'est pas possible de connaître le nombre d'itérations avant l'exécution de la boucle, on fera appel à l'une des boucles Tant Que ou Répéter.
- Pour le choix entre Tant Que et Répéter :
    - Si on doit tester la condition de contrôle avant de commencer les instructions de la boucle, on utilisera Tant Que.
    - Si la valeur de la condition de contrôle dépend d'une première exécution des instructions, on utilisera Répéter.

Chapitre. 3 :

LES PROCEDURES ET FONCTIONS

1. INTRODUCTION

Pour écrire un algorithme, l'on peut être amené à utiliser un sous-algorithme ou un sous-programme. Un sous-algorithme est un bloc d'instructions séparé du bloc principal permettant de résoudre un sous-problème. On peut appeler un sous-programme dans le programme principal ou dans un autre sous-programme.

Un sous-algorithme peut être une *FONCTION* ou une *PROCEDURE*.

2. PROCEDURE

a. Définition

Une procédure est un sous-programme ou sous-algorithme qui permet de faire une série d'action utilisable dans un autre programme.

Elle peut modifier la valeur des variables définies comme paramètres formels mais ne retourne pas de valeur. On peut afficher en outre des informations ou le contenu des variables dans une procédure.

Pour utiliser une procédure, elle doit être appelée par le programme qui veut l'utiliser. On parle d'appel de procédure.

b. Présentation

PROCEDURE nom_procédure (liste des paramètres)
[CONST Déclaration de constantes internes ou locales ;]
[VAR Déclaration de variables internes ;]
DEBUT
    {Corps de procédure}
FIN ;

c. Liste des paramètres

La définition de paramètres formels d'une procédure se présente comme suit :

(nomparamètre_1 : typeparamètre_1 ; … ; nomparamètre_n : typeparamètre_n). Les déclarations de ces paramètres sont séparées par des points-virgules. Ces paramètres peuvent

être des variables d'entrée (non modifiables), d'entrée-sortie (modifiables) ou de sortie (résultat).

d. Appel / Exécution d'une procédure

Une procédure est exécutée par le programme principal ou un autre sous-programme par appel de son nom et de ses paramètres effectifs. Il en est de même pour une fonction.

Exercices 1

1°) Ecrire une procédure qui affiche le carré d'un nombre réel saisi par l'utilisateur.

2°) Ecrire une procédure qui affiche le minimum de deux réels saisis par l'utilisateur.

Corrections Exercices 1

Exo 1

```
ALGO AFFICHE_CARRE
VAR     n : reel ;
        PROCEDURE CARRE (a : reel)
        VAR b : reel ;
        DEBUT
                b ← a * a ;
                ECRIRE ('Le carre est ', b) ;
        FIN ;
DEBUT
        ECRIRE ('Saisir un réel') ;
        LIRE (n) ;
        CARRE(n) ;
FIN ;
```

Traduction Pascal

```
PROGRAM  AFFICHE_CARRE;
VAR     n : Real ;
        PROCEDURE CARRE (a : Real);
        VAR b : Real ;
        BEGIN
                b := a * a ;
                WRITELN ('Le carre est ', b) ;
        END ;
BEGIN
        WRITELN ('Saisir un réel') ;
        READLN (n) ;
        CARRE(n) ;
END.
```

Exo 2

```
ALGO AFFICHE_MINIMUM
VAR     x, y : reel ;
        PROCEDURE MINIMUM (a, b : reel)
        VAR  Min : reel ;
        DEBUT
                Min ← a ;
                SI (b < Min) ALORS
                        Min ← b ;
                FIN SI ;
                ECRIRE ('le minimum de ', a, ' et ', b,' est ', Min) ;
        FIN ;
DEBUT
        ECRIRE ('Saisir deux réels') ;
        LIRE (x, y) ;
        MINIMUM (x, y) ;
FIN ;
```

3. FONCTION

a. Définition

A la différence d'une procédure, une fonction retourne une valeur qui sera utilisée par le programme qui appelle la fonction.

b. Présentation

```
FONCTION nom_fonction (liste des paramètres) : type_retour
CONST Déclaration-constantes-internes ;
VAR Déclaration-variables-internes ;
DEBUT
        {Corps fonction}
        Retourner (variable_retour) ; // nom_fonction← variable_retour ;
FIN ;
```

Exercices 2

1°) Ecrire une fonction qui renvoie le carré d'un réel saisi par l'utilisateur.

2°) Ecrire une fonction qui renvoie le minimum de deux réels et utiliser cette fonction dans un algorithme pour déterminer et afficher le minimum de quatre réels saisis par l'utilisateur.

3°) Ecrire la fonction qui renvoie le factoriel d'un entier inférieur à 20 et utiliser cette fonction dans un algorithme pour donner le résultat du calcul de 2n!((n-1)!+4), n étant un nombre saisi par l'utilisateur**.**

4°) Soit la fonction $f(x) = \frac{2|x-3|}{|x|}$ avec x∈ $\mathbb{R}*$: Ecrire la fonction VA qui donne la valeur absolue d'un nombre réel et utiliser cette fonction dans un algorithme pour donner la valeur de la fonction f.

Corrections Exercices 2

Exo 1

```
ALGO AFFICHE_CARRE
VAR      n : reel ;
         FONCTION CARRE (a : reel) : reel
         VAR   b : reel ;
         DEBUT
                  b ← (a * a) ;
                  CARRE← b ;
         FIN ;
DEBUT
         ECRIRE ('Saisir un réel') ;
         LIRE (n) ;
         ECRIRE ('Le carre est ', CARRE(n)) ;
FIN ;
```

Traduction Pascal

```
PROGRAM  AFFICHE_CARRE;
VAR      n : Real ;
         FUNCTION CARRE (a : Real) : Real ;
         VAR   b : Real ;
         BEGIN
                  b := (a * a) ;
                  CARRE := b ;
         END ;
BEGIN
         WRITELN ('Saisir un réel') ;
         READLN (n) ;
         WRITELN ('Le carre est ', CARRE(n)) ;
END.
```

Exo 2

```
ALGO AFFICHE_MINIMUM
VAR    a, b, c, d, Min1, Min2, Min3 : reel;
         FONCTION MINIMUM (e, f : reel) : reel
         VAR  Min : reel ;
         DEBUT
                  Min ← e ;
```

```
                SI (f < Min) ALORS
                        Min ← f ;
                FIN SI ;
                MINIMUM ← Min ;
        FIN ;
DEBUT
        ECRIRE ('Saisir 4 réels') ;
        LIRE (a, b, c, d) ;
        Min1 ← MINIMUM (a, b) ;
        Min2 ← MINIMUM (c, d) ;
        Min3 ← MINIMUM (Min1, Min2) ;
        ECRIRE ('le minimum est ', Min3) ;
FIN ;
```

Exo 3

```
ALGO CALCUL
VAR   P, R : entier ;
        FONCTION Factoriel (n : entier) : entier
        VAR   i, FACT : entier ;
        DEBUT
                FACT ← 1 ;
                POUR i DE 1 à n FAIRE
                        FACT ← FACT * i ;
                FIN POUR ;
                Factoriel← FACT ;
        FIN ;
DEBUT
        REPETER
                ECRIRE ('Saisir un entier inférieur ou égal à 20') ;
                LIRE(P) ;
        JUSQU'A  (P ≤ 20)
        FIN  REPETER ;
        R← 2 * Factoriel (P) * (Factoriel (P − 1) + 4);
        ECRIRE ('le résultat est ', R) ;
FIN ;
```

Exo 4

```
ALGO CalculValeurAbsolue
VAR   x, q : réel ;
        FONCTION VA (p : reel) : reel
        VAR    V : reel ;
        DEBUT
                SI  (p > 0) ALORS
```

```
                    V← p ;
                SINON
                    V← (−1) ∗ (p) ;
                FIN SI ;
                VA← V ;
        FIN;
DEBUT
        REPETER
                ECRIRE ('Saisir un réel différent de zéro') ;
                LIRE (x) ;
        JUSQU'A (x ≠ 0)
        FIN REPETER ;
        q ← 2 ∗ VA(x − 3)/VA (x) ;
        ECRIRE ('La valeur de la fonction f est ', q) ;
FIN ;
```

Traduction Pascal

```
PROGRAM  CalculValeurAbsolue ;
VAR     x, q : Real ;
        FUNCTION VA (p : Real) : Real;
        VAR     V : Real ;
        BEGIN
                IF  (p > 0) THEN  V := p
                ELSE  V := (−1) ∗ (p) ;
                VA← V ;
        END ;
BEGIN
        REPEAT
                WRITELN ('Saisir un réel différent de zéro') ;
                READLN (x) ;
        UNTIL (x <> 0) ;
        q :=  2 ∗ VA(x − 3)/VA (x) ;
        WRITELN ('La valeur de la fonction f est ', q) ;
END.
```

<u>Exemple de programme Pascal avec sous-programmes</u> : Températures hebdomadaires

On souhaite réaliser un programme calculant, à partir des relevés de températures sur une semaine de deux villes, laquelle a la température moyenne la plus élevée.

- Définir un type permettant de stocker les températures hebdomadaires d'une ville

```
const NBJOURS=7;
type TableauTemp=array[1..NBJOURS] of real;
```

- Écrire un sous-programme calculant la moyenne des températures de la semaine

```
function moyenne(t: TableauTemp): real;
var somme: real; i: integer;
begin
    somme := 0;
    i := 1;
    while i<= NBJOURS do
    begin
            somme := somme + t[i];
            i := i+1;
    end;
    moyenne := somme / NBJOURS;
end;
```

- Écrire un sous-programme permettant la saisie des températures hebdomadaires d'une ville

```
function jour(j:integer):string;
begin
    case j of
        1: jour:='dimanche';
        2: jour:='lundi';
        3: jour:='mardi';
        4: jour:='mercredi';
        5: jour:='jeudi';
        6: jour:='vendredi';
        7: jour:='samedi'
    end
end;
procedure saisie(var t: TableauTemp); // t précédé de var signifié que t modifiable (entrée-sortie)
var i: integer;
begin
    for i:=1 to NBJOURS do
    begin
            writeln('Temperature de ', jour(i));
            readln(t[i])
    end;
end;
```

- Écrire un sous-programme déterminant, parmi deux villes, celle qui a la température moyenne la plus élevée

```
function moyennePlusElevee(v1: string; t1: TableauTemp; v2: string; t2: TableauTemp): string;
begin
    if moyenne(t1) > moyenne(t2)
    then moyennePlusElevee := v1
    else moyennePlusElevee := v2;
end;
```

- Écrire le programme principal permettant de réaliser les traitements

```
Program TemperaturesHebdo2Villes;
const NBJOURS=7;
type TableauTemp=array[1..NBJOURS] of real;
{Insérer les differents sous-programmes precedents dans l'ordre}
var t1, t2: TableauTemp;
    v1, v2: string;
begin
    writeln('Ville 1');
    readln(v1);
    saisie(t1);
    writeln('Ville 2');
    readln(v2);
    saisie(t2);
    write(moyennePlusElevee(v1,t1,v2,t2),' a une temperature plus elevee');
end.
```

Exercices 3 : QCM (un seul item correct par question)

1. La réutilisabilité d'un algorithme est son aptitude
(a) à utiliser de manière optimale les ressources du matériel qui l'exécute
(b) à se protéger de conditions anormales d'utilisation
(c) à résoudre des tâches équivalentes à celle pour laquelle il a été conçu
(d) à réaliser exactement la tâche pour laquelle il a été conçu

2. Une fonction est un bloc d'instructions nommé et paramétré
(a) qui ne peut pas retourner plusieurs valeurs
(b) qui ne peut pas contenir d'instructions itératives
(c) qui retourne une valeur
(d) qui ne retourne pas de valeur

3. Les paramètres d'entrée d'une fonction sont
(a) les arguments nécessaires pour effectuer le traitement associé à la fonction
(b) les valeurs obtenues après avoir effectué le traitement associé à la fonction
(c) des grandeurs invariantes pendant l'exécution de la fonction
(d) des variables auxiliaires définies dans le corps de la fonction

4. Les préconditions d'une fonction sont des conditions à respecter
(a) par les paramètres de sortie de la fonction
(b) pendant toute l'exécution de la fonction
(c) par les paramètres d'entrée de la fonction
(d) pour pouvoir compiler la fonction

5. La description d'une fonction décrit
(a) ce que fait la fonction
(b) comment fait la fonction
(c) pourquoi la fonction le fait
(d) où la fonction le fait

6. Le jeu de tests d'une fonction est
(a) un ensemble d'exercices à résoudre
(b) un ensemble d'exceptions dans le fonctionnement de la fonction
(c) un ensemble de caractéristiques d'entrées-sorties associées
(d) un ensemble de recommandations dans l'utilisation de la fonction

7. La validité d'une fonction est son aptitude à réaliser exactement la tâche pour laquelle elle a été conçue. Plus concrètement,
(a) la fonction doit vérifier impérativement ses préconditions
(b) la fonction doit être correctement paramétrée
(c) l'implémentation de la fonction doit être conforme aux jeux de tests
(d) l'utilisation de la fonction doit être conviviale

8. Un appel récursif est un appel
(a) dont l'exécution est un processus récursif
(b) dont l'exécution est un processus itératif
(c) dont le résultat est retourné par la fonction
(d) d'une fonction par elle-même

Chapitre. 3 :

LES TABLEAUX A UNE DIMENSION

1. GENERALITES

a. Définition

Un tableau est une structure de données qui permet de mémoriser et de manipuler un ensemble d'éléments de même type.

b. Déclaration de variable de type tableau

- Déclaration directe

VAR t : tableau [$valdeb..valfin$] de type_élément ;
valdeb et valfin sont en général des entiers ; mais peut être aussi des caractères. Ils sont les indices d'accès aux différentes valeurs du tableau.

Exemple
VAR t : tableau [1..20] d'entier ;
permet de déclarer un tableau de 20 entiers.

- Déclaration indirecte

TYPE tab = tableau [$valdeb..valfin$] de type_élément ;
VAR t : tab ;

c. Accès à un élément d'un tableau.

Soit un tableau **t**, pour accéder au **i-ème** élément du tableau ou à l'élément à la position **i**, on utilise la variable **t**[**i**].

d. Affectation d'une valeur à un élément d'un tableau.

Pour affecter la valeur **V** au **i-ème** élément du tableau **t**, on fait : **t**[i] ← **V** ;

Exercices d'application

1) Ecrire un algorithme qui affecte à un tableau de 20 entiers respectivement les 20 premiers entiers naturels.

2) Ecrire un algorithme qui affecte à un tableau de 10 entiers respectivement le carré des 10 premiers entiers naturels non nuls.

3) Ecrire un algorithme qui demande à l'utilisateur de renseigner un tableau de 10 réels et qui lui affiche le minimum et le maximum.

Corrections

Exo 1

```
ALGO  TABLEAU
VAR   T : tableau [1..20] d'entier ;
      i : entier ;
DEBUT
      POUR  i de 1 à 20 FAIRE
            T[i] ← i − 1 ;
            ECRIRE (T[i]) ;
      FIN POUR ;
FIN ;
```

Traduction Pascal

```
PROGRAM  TABLEAU ;
VAR   T : ARRAY[1..20] of Integer ;
      i : Integer ;
BEGIN
      FOR  i := 1 TO 20 DO
      BEGIN
            T[i] ≔  i − 1 ;
            WRITELN (T[i]) ;
      END ;
END.
```

Exo 2

```
ALGO TABLEAU
VAR    T : tableau[1..10] d'entier ;
       i : entier ;
DEBUT
       POUR i de 1 à 10 FAIRE
             T[i] ← i ∗ i;
             ECRIRE (T[i]) ;
       FIN POUR ;
FIN ;
```

Traduction Pascal

```
PROGRAM  TABLEAU ;
VAR    T : ARRAY[1..10] of Integer ;
       i : Integer ;
BEGIN
       FOR i := 1 TO 10 DO
       BEGIN
              T[i] := i * i;
              WRITELN (T[i]) ;
       END ;
END.
```

Exo 3

```
ALGO minimum_maximum
VAR   T : tableau[1..10] de reel ;
       Min, Max : reel ;
       i : entier ;
DEBUT
       POUR  i de 1 à 10 FAIRE
              ECRIRE ('Entrer le réel ', i) ;
              LIRE (T[i] ) ;
       FIN POUR ;
       Min← T[1] ;
       Max← T[1] ;
       POUR  i de 2 à 10 FAIRE
              SI ← T[i] < Min  ALORS
                     Min ← T[i] ;
              FIN  SI ;
              SI ← T[i] > Max  ALORS
                     Max ←  T[i] ;
              FIN  SI ;
       FIN  POUR ;
       ECRIRE ('Les éléments sont') ;
       POUR  i de 1 à 10  FAIRE
              ECRIRE (T[i]) ;
       FIN POUR ;
       ECRIRE ('Le minimum est : ', Min, ' et le maximum est : ', Max) ;
FIN ;
```

2. OPERATIONS SUR LES TABLEAUX NUMERIQUES

Les opérations sur les tableaux sont les opérations applicables aux types d'éléments de ces tableaux.

a. Somme des tableaux de réels ou d'entiers.

Soient deux tableaux T1 et T2. Le tableau T3 leur somme est définie par :

$T3[i] \leftarrow T1[i] + T2[i]$; avec T1 et T2 de même dimension.

On peut aussi faire la somme des éléments d'un tableau avec une valeur fixe donnée.

$T3[i] \leftarrow T1[i] + v$;

b. Soustraction ou différence de tableaux

$T3[i] \leftarrow T1[i] - T2[i]$;

$T3[i] \leftarrow T1[i] - v$;

c. Produit de tableaux

$T3[i] \leftarrow T1[i] * T2[i]$;

$T3[i] \leftarrow T1[i] * v$;

d. Rapport de tableaux

$T3[i] \leftarrow \frac{T1[i]}{T2[i]}$; avec $T2[i] \neq 0$,

$T3[i] \leftarrow \frac{T1[i]}{v}$; avec v$\neq$ 0

Exercice d'application

Ecrire un algorithme qui demande à l'utilisateur de saisir 10 réels positifs qui l'affecte à un tableau et qui lui renvoi la moyenne des éléments du tableau.

Correction

```
ALGO MoyenneTableau
VAR   T : tableau[1..10] de reel ;
        S, Moy : reel ;
        i : entier ;
 DEBUT
        S← 0;
        POUR i de 1 à 10 FAIRE
                ECRIRE ('Saisir le réel ; ' i) ;
                LIRE ((T[i]) ;
                TANT QUE ((T[i] ≤ 0) FAIRE
                        ECRIRE ('Saisir le réel' ; i) ;
                        LIRE ((T[i]) ;
                FIN TANT QUE ;
                S← S + (T[i];
        FIN POUR ;
        Moy ←S/10 ;
        ECRIRE ('La moyenne est ', Moy) ;
FIN ;
```

Traduction Pascal

```
PROGRAM MoyenneTableau ;
VAR   T : ARRAY[1..10] of Real ;
        S, Moy : Real ;
        i : Integer ;
BEGIN
        S← 0;
        FOR i := 1 TO 10 DO
        BEGIN
                WRITELN ('Saisir le réel ; ' i) ;
                READLN ((T[i]) ;
                WHILE ((T[i] ≤ 0) DO
                BEGIN
                        WRITELN ('Saisir le réel' ; i) ;
                        READLN ((T[i]) ;
                END ;
                S := S + (T[i];
        END ;
        Moy := S/10 ;
        WRITELN ('La moyenne est ', Moy) ;
END.
```

3. TRI D'UN TABLEAU

a. Tri par sélection

```
ALGO TRI_SELECTION
VAR     T : tableau [1..n] de réel ;
        Tempo : reel ;
        i, j, Posmin : entier ;
DEBUT
  POUR i de 1 à (n-1) FAIRE
        Posmin← i;
        POUR j de (i+1) à n FAIRE
              SI T[j] < T[Posmin] ALORS
                    Posmin← j;
              FIN SI ;
        FIN POUR ;
        SI (Posmin≠ i) ALORS
              Tempo← T[Posmin] ;
              T[Posmin] ← T[i];
              T[i] ← Tempo ;
        FIN SI ;
  FIN POUR ;
FIN ;
```

b. Tri par insertion

```
ALGO TRI-INSERTION
VAR   i, j, n : entier ;
        T : tableau[1..n] de reel;
        X : reel ;
DEBUT
  POUR i de 2 à n FAIRE
        X← T[i];
        j← i − 1;
        TANT QUE (j> 0) ET T[j] > X)FAIRE
              T[j + 1] ← T[j] ;
              J← j − 1;
        FIN TANT QUE ;
        T[j + 1] ← X;
  FIN POUR ;
FIN ;
```

4. RECHERCHE DICHOTOMIQUE

La recherche dichotomique ou recherche par dichotomie se fait sur les éléments d'un tableau préalablement trié.

```
ALGO RECHERCHE_DICHOTOMIQUE
VAR   n, Sup, Inf, milieu : entier
        T : tableau[1..n] de reel ;
        Fin : booléen ;
        Val : reel ;
DEBUT
        ECRIRE ('Saisir la valeur recherchée') ;
        LIRE (Val) ;
        Sup← n;
        Inf ← 1;
        Fin← Faux;
        TANT QUE (Fin=Faux) FAIRE
                Milieu← (Sup + Inf) ÷ 2;
                SI Val< T[milieu] ALORS
                        Sup← milieu − 1;
                SINON
                        Inf ← milieu + 1;
                FIN SI ;
                Fin← [(Val = T[milieu] ou (Sup < Inf)] ;
        FIN TANT QUE ;
        SI (Val= T[milieu]) ALORS
                ECRIRE (Val, 'est dans le tableau') ;
        FIN SI ;
FIN ;
```

Chapitre.4 :

LES TABLEAUX A DEUX DIMENSIONS OU MATRICES

1. GENERALITES

a. Définition

Tout comme un tableau à une dimension, un tableau bidimensionnel ou une matrice est une structure de données permettant de mémoriser et de manipuler un ensemble d'éléments de même type.

b. Déclaration de variable de type matrice

- Première méthode

VAR Tab : tableau[1..m, 1..n] de type_élément ;
avec m : nombre de colonne et n le nombre de ligne

Exemple

VAR t : tableau [1..30, 1..10] de reel ;
t est un tableau de 30 colonnes et 10 lignes comportant des réels.

- Deuxième méthode

TYPE tab = tableau[1..m] de type_élément ;
VAR T : tableau[1..n] de tab ;
avec m : nombre de colonne et n le nombre de ligne

Exemple

TYPE tab1 = tableau [1..30] de reel ;
VAR t1 : tableau [1..10] de tab1 ;
t1 est un tableau de 30 colonnes et 10 lignes comportant des réels.

c. Accès à un élément d'une matrice

Soit la matrice t1 de dimension 10x15 (10 lignes et 15 colonnes).
Pour accéder à un élément de position i, j de ce tableau, on fait **t1[i,j]**.

d. Affectation d'une valeur à un élément d'une matrice

Pour affecter une valeur **V** à un élément de position i, j d'une matrice m1, on fait : **m1[i,j] ←V ;**

2. OPERATIONS SUR LES MATRICES DE NUMERIQUES

Soient deux matrices m1 et m2 de mêmes dimensions et V une valeur donnée.

a. Somme d'une valeur à une matrice

m1[i,j] ← $m1[i,j] + V$; signifie que les m1[i,j] sont augmentés de la valeur V.

b. Soustraction d'une valeur à une matrice

m1[i,j] ← $m1[i,j] - V$; signifie que les m1[i,j] sont diminués de la valeur V.

c. Multiplication d'une matrice par une valeur

m1[i,j] ← $m1[i,j] * V$; signifie que les m1[i,j] sont multipliés par la valeur V.

d. Division d'une matrice par une valeur

m1[i,j] ← $m1[i,j] / V$; avec $V \neq 0$; signifie que les m1[i,j] sont divisés par la valeur V.

e. Somme de matrices

m1[i,j] ← $m1[i,j] + m2[i,j]$; signifie que les m1[i,j] sont augmentés de m2[i,j] respectivement de même rang.

f. Soustraction de matrices

m1[i,j] ← $m1[i,j] - m2[i,j]$; signifie que les m1[i,j] sont diminués de m2[i,j] respectivement de même rang.

SERIE DE TD

PARTIE I

a. Copier ou couper et coller sous Windows : les différentes formes et leurs particularités ou conséquences.

b. Suppression d'un fichier ou dossier sous Windows : les différentes formes et leurs particularités ou conséquences.

c. Les différentes sortes de logiciels.

d. L'histoire de l'ordinateur.

e. L'histoire de Facebook.

f. Qu'est ce Whatsapp ? Donnez ces avantages et inconvénients.

PARTIE II

1) Soit un tableau de 30 mots (chaînes de caractères). Soient deux mots mot1 et mot2 saisis par l'utilisateur. Ecrire un algorithme permettant de remplacer dans ce tableau toute les occurrences ou apparitions du mot1 par le mot2. L'algorithme doit aussi permettre d'afficher toutes les positions de remplacement.

2) Soit un tableau T de 50 réels. Ecrire un algorithme qui permet à l'utilisateur d'entrer les valeurs des éléments du tableau, de vérifier la présence d'un nombre réel V saisi par ce dernier parmi les éléments du tableau et d'afficher le nombre d'occurrences de V s'il en existe.

3) Soit une suite numérique définie par : $Un\text{-}1=\frac{1}{2}Un\text{-}3Un\text{-}2$ avec U1=0 et U2=1

Ecrire un algorithme qui déclare un tableau de 10 entiers, qui renseigne les éléments du tableau avec les différents termes de la suite numérique, affiche les éléments du tableau et renvoi comme résultat la somme des éléments du tableau.

4) Proposer un algorithme qui affecte les valeurs des éléments d'un tableau de 15 entiers suivant la suite de Fibonacci et qui ensuite affiche ces valeurs (les deux premiers termes de cette suite étant égaux à 1).

5) Soient deux tableaux de 50 entiers. Ecrire un algorithme qui affiche « vrai » si les deux tableaux possèdent au moins une valeur identique de même rang, sinon il affiche « faux »

6) Soit le pseudo triangle de Pascal suivant :

```
1
1    5
1    5   11
1    5   11   19
1    5   11   19   29
1    5   11   19   29   41
```

Ecrire l'algorithme dont l'exécution de son programme en langage Pascal donne ce résultat.

TAF pour la partie II : Traduire les algorithmes écrits en programme avec le langage Pascal. Exécuter ces programmes et faire les captures d'écran des résultats de l'exécution.

BIBLIOGRAPHIE

Cours Algorithmique et Programmation JAVA de Gabès 2010-2011